

Appendices: the scripts

The functions

A lot of computations are repeated in the different scripts, so this is useful to write functions in a complementary script. We will explain in this annex some of those functions. Sometimes, there are different functions for whether there is a binary or merger file.

First, there are functions to compute the radii containing different percentages of either the number of objects, the total mass or the total light. You have an example of the first function to give an idea as the Listing 1.

```

1 def radius_number(percent, distance, N_tot):
2     distance.sort()
3     R_frac_n=[]
4     for frac in percent:
5         N_percent=int(N_tot*frac)-1 #the number of stars that must be
        contained in this radius
6         R_frac_n.append(distance[N_percent])
7     return R_frac_n

```

Listing 1: Python function computing the radius at different percentages of number of objects

- *percent* is a list containing the percentages as floats that we want
- *distance* is the list with the distance from the center of every object or system
- *N_tot* is the total number of objects in the cluster
- *R_frac_n* is a list of the radii containing the different percentages of number of objects

Then there are functions to count how many of a CO type are in different systems: single, binary, CO-CO binary, ternary, CO-CO-else ternary or CO-CO-CO ternary; and how many in the half-mass radius, and the total mass represented by those type of CO. They only need *k*, the number stating to the type of object it is (*K** in the description of the file), the table(s) containing the informations of each objects and the number of rows in the table(s). There is an example of the function counting the number of binary systems containing a CO of type *k* as Listing 2.

```

1 def countbinary(k, binary, N_binary, star, WD):
2     #count how many binaries with at least one star of type k
3     #and give number of the type of the companion [star, WD, NS, BH]
4     companion=[0, 0, 0, 0]
5     N=0
6     for i in range(N_binary):
7         if binary['K*1'][i]==k:
8             N+=1
9             if binary['K*2'][i]==14:
10                companion[3]+=1
11            elif binary['K*2'][i]==13:
12                companion[2]+=1
13            for K in WD:

```

```

14         if binary['K*2'][i]==K:
15             companion[2]+=1
16     for K in star:
17         if binary['K*2'][i]==K:
18             companion[2]+=1
19     elif binary['K*2'][i]==k:
20         N+=1
21         if binary['K*1'][i]==14:
22             companion[3]+=1
23         elif binary['K*1'][i]==13:
24             companion[2]+=1
25     for K in WD:
26         if binary['K*1'][i]==K:
27             companion[2]+=1
28     for K in star:
29         if binary['K*1'][i]==K:
30             companion[2]+=1
31     return (N, companion)

```

Listing 2: Python function computing the number of binary systems containing a CO of type k and giving the the number of companions of each type

Then, there are functions to compute the distributions of the parameters listed in the section 2.1. There are three functions to loop on the bins, differing in the type of file there is for this snapshot (single, binary, merger), that take as input a function that computes the value for one single bin (see example Listing 3) and array(s) not containing every informations given in the files. There is a list per file and there is, the distance from the center of the cluster, the mass of the object, its luminosity, its coordinates and its velocity coordinates, in that order. If it is a system, some pieces of information are doubled; for example, for binary, there is the distance of the system, then the mass of the first object, the mass of the second object, the luminosity of the first object, etc.

```

1 def distribution_single(distri_function_single, single):
2     distribution=[]
3     radii=[]
4     sing=0
5     i=0
6     Nsingle=len(single)
7     Nbin=bins(Nsingle)
8     loop=Nsingle/Nbin
9
10    while i<loop and sing<Nsingle:
11        r, d, s = distri_function_single(Nbin, sing, single)
12        distribution.append(d)
13        radii.append(np.mean(r))
14        sing=s
15        i+=1
16    return(radii, distribution, Nbin)

```

Listing 3: Python function computing a distribution

with for example as `distri_function_single` the following function to compute the mass:

```

1 def mass_single(N, s, single):
2     distribution=0
3     radii=[]

```

```

4     i=0
5     sing=s
6
7     while i<N and sing<len(single):
8         distribution+=single[sing][1]
9         radii.append(single[sing][0])
10        sing+=1
11        i+=1
12    return(radii, distribution, sing)

```

Listing 4: Python function computing the mass of a bin

Notice that the binning of the distribution is defined by a function. This function is quite arbitrary and written thanks to trials and errors as Listing 5. It depends on the length of the list, as it is quite variable all along the code; the list of BH is much shorter than the list of all the objects, therefore we need different binnings to have readable curves, and even avoid some code error.

```

1 def bins(Nliste):
2     if Nliste//100000!=0:
3         Nbin=Nliste//100000*4000
4     elif Nliste//10000>4:
5         Nbin=4000
6     elif Nliste//10000!=0:
7         Nbin=2000
8     elif Nliste//1000>4:
9         Nbin=400
10    elif Nliste//1000!=0:
11        Nbin=180
12    elif Nliste//100>4:
13        Nbin=60
14    else:
15        Nbin=30
16    return Nbin

```

Listing 5: Python function computing the bin for a specific list

There is also a function like Listing 3 that computes a higher resolution at the beginning and the end of the curve. Fundamentally, it recreates the distribution of Listing 3, except it replaces the first and last bin by 150.

```

1 def highres_single(distri_function_single, single):
2     distribution=[]
3     radii=[]
4     sing=0
5     Nsingle=len(single)
6     Nbin=bins(Nsingle)
7     loop=Nsingle/Nbin
8
9     nbin=Nbin/150 #not always possible !!
10    for j in range(150):
11        r, d, s = distri_function_single(nbin, sing, single)
12        distribution.append(d)
13        radii.append(np.mean(r))
14        sing=s
15
16    i=1

```

```

17 while i<loop-1 and sing<Nsingle:
18     r, d, s = distri_function_single(Nbin, sing, single)
19     distribution.append(d)
20     radii.append(np.mean(r))
21     sing=s
22     i+=1
23
24 while sing<Nsingle:
25     r, d, s = distri_function_single(nbin, sing, single)
26     distribution.append(d)
27     radii.append(np.mean(r))
28     sing=s
29     i+=1
30
31 return(radii, distribution, Nbin)

```

Listing 6: Python function computing a distribution with high resolution at the beginning and end

Among those distributions, as explained in the first section, we want the triaxiality and ellipticity. Those were complex because we had to verify, among I_{xx} , I_{yy} and I_{zz} the order between them, before computing the value for a bin. That is why we needed the Listing 7. Therefore the function needs the distribution of the different I_{ii} , in a list containing them in order, and it returns the 2 distributions.

```

1 def eccentricity(Iii):
2     et=[]
3     ed=[]
4     for i in range(len(Iii)):
5         if Iii[i][1]>Iii[i][2] and Iii[i][2]>Iii[i][0]:
6             et.append((Iii[i][1]**2-Iii[i][2]**2)/(Iii[i][1]**2-Iii[i]
7 ] [0]**2))
8             ed.append(1-Iii[i][0]/Iii[i][1])
9         elif Iii[i][1]>Iii[i][0] and Iii[i][0]>Iii[i][2]:
10            et.append((Iii[i][1]**2-Iii[i][0]**2)/(Iii[i][1]**2-Iii[i]
11 ] [2]**2))
12            ed.append(1-Iii[i][2]/Iii[i][1])
13        elif Iii[i][0]>Iii[i][1] and Iii[i][1]>Iii[i][2]:
14            et.append((Iii[i][0]**2-Iii[i][1]**2)/(Iii[i][0]**2-Iii[i]
15 ] [2]**2))
16            ed.append(1-Iii[i][2]/Iii[i][0])
17        elif Iii[i][0]>Iii[i][2] and Iii[i][2]>Iii[i][1]:
18            et.append((Iii[i][0]**2-Iii[i][2]**2)/(Iii[i][0]**2-Iii[i]
19 ] [1]**2))
20            ed.append(1-Iii[i][1]/Iii[i][0])
21        elif Iii[i][2]>Iii[i][1] and Iii[i][1]>Iii[i][0]:
22            et.append((Iii[i][2]**2-Iii[i][1]**2)/(Iii[i][2]**2-Iii[i]
23 ] [0]**2))
24            ed.append(1-Iii[i][0]/Iii[i][2])
25        elif Iii[i][2]>Iii[i][0] and Iii[i][0]>Iii[i][1]:
26            et.append((Iii[i][2]**2-Iii[i][0]**2)/(Iii[i][2]**2-Iii[i]
27 ] [1]**2))
28            ed.append(1-Iii[i][1]/Iii[i][2])
29    return (et, ed)

```

Listing 7: Python function computing each value of triaxiality and ellipticity

As you can tell, the function in listing 3 needs a list (or some lists if there are binary or ternary systems) with its parameter in a specific order. When computing the distribution for a specific population (a type of object or objects within a specific mass range), we need to create this list so we don't have programming errors. Once again, there are three different functions that depend on the files of the snapshot, because either we just need to create a list, or we need to couple either three (Listing 8) or six lists (the list of singles, the list of first objects in the binary file, the list of second objects in the binary file, etc)

```

1 def coupling(m1, m2, m3):
2     #couples m1, m2, m3 in one list sorted by first column
3     liste=[]
4     if len(m1)>1:
5         for i in range(len(m1)):
6             item=[m1[i][0], m1[i][1], m1[i][2], m1[i][3], m1[i][4], m1[i]
7 ] [5], m1[i][6], m1[i][7], m1[i][8]]
8             liste.append(item)
9     elif len(m1)==1:
10        item=[m1[0][0], m1[0][1], m1[0][2], m1[0][3], m1[0][4], m1
11 ] [5], m1[0][6], m1[0][7], m1[0][8]]
12        liste.append(item)
13    if len(m2)>1:
14        for i in range(len(m2)):
15            item=[m2[i][0], m2[i][1], m2[i][2], m2[i][3], m2[i][4], m2[i]
16 ] [5], m2[i][6], m2[i][7], m2[i][8]]
17            liste.append(item)
18    elif len(m2)==1:
19        item=[m2[0][0], m2[0][1], m2[0][2], m2[0][3], m2[0][4], m2
20 ] [5], m2[0][6], m2[0][7], m2[0][8]]
21        liste.append(item)
22    if len(m3)>1:
23        for i in range(len(m3)):
24            item=[m3[i][0], m3[i][1], m3[i][2], m3[i][3], m3[i][4], m3[i]
25 ] [5], m3[i][6], m3[i][7], m3[i][8]]
26            liste.append(item)
27    elif len(m3)==1:
28        item=[m3[0][0], m3[0][1], m3[0][2], m3[0][3], m3[0][4], m3
29 ] [5], m3[0][6], m3[0][7], m3[0][8]]
30        liste.append(item)
31    liste.sort()
32    return(liste)

```

Listing 8: Python function coupling three lists

We also said in the first and second section that we needed the value of a lot of parameters for half the objects in the considered population. When considering all objects in the cluster, we solely need to use the functions `distri_function_single` (or their binary/merger version) with a binning of $\frac{N_{tot}}{2}$. When computing the distributions for a list of populations (per mass range or type of object); to be able to loop easily, we created a list containing those populations and naming it *massdist*. Therefore, *massdist*[*j*] is the list of BHs for example.

```

1 def half_discri(massdist, m):
2     Rh_n=[]
3     Rh_m=[]
4     for j in range(m):

```

```

5     ntot=len(massdist[j])
6     if ntot>0:
7         mtot=sum([massdist[j][i][1] for i in range(ntot)])
8         i_n=int(ntot/2)
9         Rh_n.append(massdist[j][i_n][0])
10        i_m, mass =0, 0
11        while mass<=mtot/2 and i_m<ntot:
12            mass+=massdist[j][i_m][1]
13            i_m+=1
14        Rh_m.append(massdist[j][i_m-1][0])
15    else:
16        Rh_n.append(0)
17        Rh_m.append(0)
18    mdens50=[]
19    ml50=[]
20    sigma50=[]
21    mean50=[]
22    ecc50=[]
23    ini50=[]
24    for j in range(m):
25        if len(massdist[j])>0:
26            rd, dist, s = mass_single(len(massdist[j])/2, 0, massdist[j
27            ])
28            r, lu, s = lum_single(len(massdist[j])/2, 0, massdist[j])
29            rc, cou, s = count_single(len(massdist[j])/2, 0, massdist[j
30            ])
31            r, v, s = velocities_single(len(massdist[j])/2, 0, massdist[
32            j])
33            r, Iii, s = Inertia_single(len(massdist[j])/2, 0, massdist[j
34            ])
35
36            mdens50.append(dist/(np.mean(rd)**3*4/3*np.pi))
37            ml50.append(dist/lu)
38            sigmar50, sigmap50, sigmat50 = np.std(v[0]), np.std(v[1]),
39            np.std(v[2])
40            beta50=1-(pow(sigmap50, 2)+pow(sigmat50, 2))/(2*pow(sigmar50
41            , 2))
42            sigma50.append([sigmar50, sigmap50, sigmat50, beta50])
43            meanr50, meanp50, meant50 = np.mean(v[0]), np.mean(v[1]), np
44            .mean(v[2])
45            mean50.append([meanr50, meanp50, meant50])
46            et, ed = eccentricity([Iii])
47            ecc50.append([et, ed])
48            ini50.append(Iii)
49        else:
50            mdens50.append(0)
51            ml50.append(0)
52            sigma50.append([0, 0, 0, 0])
53            mean50.append([0, 0, 0])
54            ecc50.append([0], [0])
55            ini50.append([0, 0, 0])
56
57    return(Rh_n, Rh_m, mdens50, ml50, sigma50, mean50, ecc50, ini50)

```

Listing 9: Python function computing the value of defined parameters for half the population of massdist

Finally, we need functions to compute the spacial distribution of types of objects for subsection 2.3. If we want a spacial distribution of mass, we need to add weights to the histogram that are the mass of the objects. Once again, there are three different functions, because in the histograms, we differentiate between the types of system.

```

1 def distri_position_single(k, single, N_single):
2     #give the position of all stars of type k
3     singles=[]
4     weights=[]
5     for i in range(N_single):
6         if single['K*'][i]==k:
7             singles.append(single['dist[pc]'][i])
8             weights.append(single['M[M*]'][i])
9     return (singles, weights)

```

Listing 10: Python function computing the list of positions of objects of type k and their mass

The main script to analyse a snapshot

A lot is done in this script. This is where all the data files are written. The first one is a *.txt* file that is meant to be read by humans, with general information, such as the one put in tables for comparison in this report. This file is named "output" followed by the number of the snapshot and it is open during all the script.

Then, we open the snapshot files and create ascii tables from the astropy module. We always have a "single" file, but we need to test if we have "binary" and "merger" files. At the same time, we create boolean variables BINARY and MERGER, to know all along the code which function to use. For each table created, we rename the columns as the corresponding parameter to ease the process. We also create a new column for each table named 'dist[pc]', where we compute the distance from the center of the cluster from the Cartesian coordinates that are given.

The files are different whether they are from the level A or C simulation. The level C simulation has particularities. First, there are some duplicates between the "single" and the "binary" files, noted as -1 in the second column of the single file (other objects are noted as 0 in this column), so we need to delete them in one of the tables created. We decided to delete them from the single file because it is easier thanks to the second column, and we would loose some information if we deleted in the binary table, but not in the single table. Another particularity in this level, is that it needed to be done in 2 times, so the second half of the snapshots is in a different file, and they have a different "global" file.

Then we compute the radii containing different percentages of either the number of objects, the total mass or the total light, using the functions described in the first annex. We imported the script of the latter as *func*.

```

1 percent=[0.01, 0.2, 0.5, 0.9, 1.0] #the percentages for the lagrangian
    radii we want to compute
2 file.write(f"fractions: {percent} \n")
3 R_frac_n=func.radius_number(percent, distance, N_tot)
4 file.write(f"number R_frac= {R_frac_n} \n")
5
6 if MERGER:

```

```

7     R_frac_m=func.radius_mass_merger(percent, M_tot, single, N_single,
    binary, N_binary, merger, N_merger)
8     R_frac_l=func.radius_lum_merger(percent, single, N_single, binary,
    N_binary, merger, N_merger)
9 elif BINARY:
10    R_frac_m=func.radius_mass_binary(percent, M_tot, single, N_single,
    binary, N_binary)
11    R_frac_l=func.radius_lum_binary(percent, single, N_single, binary,
    N_binary)
12 else:
13    R_frac_m=func.radius_mass_single(percent, M_tot, single, N_single)
14    R_frac_l=func.radius_lum_single(percent, single, N_single)
15
16 file.write(f"mass R_frac= {R_frac_m} \n")
17 file.write(f"light R_frac= {R_frac_l} \n")

```

Listing 11: Python script computing lists of radii thanks to functions

Then, before being able to count how many of a CO type are in different systems: single, binary, CO-CO binary, ternary, CO-CO-else ternary or CO-CO-CO ternary; and how many in the half-mass radius, and the total mass represented by those types of CO, we need to define some types. The code is defined as, for example:

- $K^* = 1$, the object is a Main Sequence star
- $K^* = 3$, the object is Red Giant
- $K^* = 11$, the object is a Carbon-Oxygen WD
- $K^* = 12$, the object is a Oxygen-Neon WD
- $K^* = 13$, the object is a Neutron star.
- $K^* = 14$, the object is a Black hole.

So we needed to group the different types of stars and WDs in a list to loop on them when functions need a number to work such as in Listing 12.

```

1 star=[-1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 WD=[10, 11, 12]
3
4 N_MS=sum([func.countsingle(k, single, N_single) for k in star])
5 N_WD=sum([func.countsingle(k, single, N_single) for k in WD])
6 N_NS=func.countsingle(13, single, N_single)
7 N_BH=func.countsingle(14, single, N_single)

```

Listing 12: Python script computing the number of each type of objects that are singles thanks to functions

Then we compute the different distributions plotted in the first section. At the same time, we compute the value for half the objects as shown in Listing 13. Then the informations are written in

```

1 elif BINARY:
2     lsingle=single['dist[pc]', 'M[M*]', 'L[L*]', 'x[pc]', 'y[pc]', 'z[pc]
    ', 'vx[km/s]', 'vy[km/s]', 'vz[km/s]']

```

```

3     lbinary=binary['dist[pc]', 'M1[M*]', 'M2[M*]', 'L1[L*]', 'L2[L*]', '
xCM[pc]', 'yCM[pc]', 'zCM[pc]', 'vxCM[km/s]', 'vyCM[km/s]', 'vzCM[km/
s]']
4
5     rd, dist, s, b = func.mass_binary(N_tot/2, 0, 0, lsingle, lbinary)
6     r, lu, s, b = func.lum_binary(N_tot/2, 0, 0, lsingle, lbinary)
7     rc, cou, s, b = func.count_binary(N_tot/2, 0, 0, lsingle, lbinary)
8     r, v, s, b = func.velocities_binary(N_tot/2, 0, 0, lsingle, lbinary)
9     r, ini, s, b = func.Inertia_binary(N_tot/2, 0, 0, lsingle, lbinary)
10
11    radid, distribution, nd = func.distribution_binary(func.mass_single,
func.mass_binary, lsingle, lbinary)
12    radil, lum, nl = func.distribution_binary(func.lum_single, func.
lum_binary, lsingle, lbinary)
13    radic, count, nc = func.distribution_binary(func.count_single, func
.count_binary, lsingle, lbinary)
14    radis, velocities, nv = func.distribution_binary(func.
velocities_single, func.velocities_binary, lsingle, lbinary)
15    radii, Iii, ni = func.distribution_binary(func.Inertia_single, func.
Inertia_binary, lsingle, lbinary)
16
17    mdens50=dist/(np.mean(rd)**3*4/3*np.pi)
18    dens50=cou/(np.mean(rc)**3*4/3*np.pi)
19    ml50=dist/lu
20    sigmar50, sigmap50, sigmat50 = np.std(v[0]), np.std(v[1]), np.std(v[2])
21    beta50=1-(pow(sigmap50, 2)+pow(sigmat50, 2))/(2*pow(sigmar50, 2))
22    meanr50, meanp50, meant50 = np.mean(v[0]), np.mean(v[1]), np.mean(v[2])
23    et50, ed50 = func.eccentricity([ini])
24
25    mdensity=[distribution[0]/(radid[0]**3*4/3*np.pi)]
26    mdensity=mdensity+[distribution[i]/((4/3*np.pi*radid[i]**3)-(4/3*np.pi*
radid[i-1]**3)) for i in range(1, len(radid))]
27    ML=[distribution[i]/lum[i] for i in range(len(radid))]
28
29    density=[count[0]/(radic[0]**3*4/3*np.pi)]
30    density=density+[count[i]/((4/3*np.pi*radic[i]**3)-(4/3*np.pi*radic[i
-1]**3)) for i in range(1, len(radic))]
31
32    sigma=[[np.std(velocities[i][j]) for i in range(len(radis))] for j in
range(3)]
33    beta=[1-(pow(sigma[1][n], 2)+pow(sigma[2][n], 2))/(2*pow(sigma[0][n], 2)
) for n in range(len(radis))]
34    meanv=[[np.mean(velocities[i][j]) for i in range(len(radis))] for j in
range(3)]
35
36    I2=[[Iii[i][j] for i in range(len(radii))] for j in range(3)]
37    et, ed = func.eccentricity(Iii)

```

Listing 13: Python script computing the distributions and half values if there is only a single and a binary files

Once they are computed, we can write the values in a file to use them outside this script. Because we know we will run this code more than 3 000 times, we decided not to write a file per snapshot, but a file per type of data, where each line corresponds to a snapshot. Therefore, when opening the file to write in it, we mustn't use the "w" (for write) parameter but "a" (for append), to add to the file if there is already something

written and not delete it.

Now we can do the same computing of distributions, but for different populations. For example, we separated the objects per mass range in the Listing 14, if we have binary systems. We can do the same thing for the other tables of courses, and if we want to separate them by type of objects, we replace what is between parentheses of the numpy function "where" with a condition on the 'K*' variable.

```

1 ranges=[0.0, 0.2, 0.3, 0.4, 0.6, 0.9]
2 m=len(ranges)
3
4 massdist=[]
5 for j in range(1, m):
6     m1=single['dist[pc]', 'M[M*]', 'L[L*]', 'x[pc]', 'y[pc]', 'z[pc]', '
       vx[km/s]', 'vy[km/s]', 'vz[km/s]'][(np.where((single['M[M*]']>ranges[
       j-1]) & (single['M[M*]']<ranges[j])))]
7     m2=binary['dist[pc]', 'M1[M*]', 'L1[L*]', 'xCM[pc]', 'yCM[pc]', 'zCM
       [pc]', 'vxCM[km/s]', 'vyCM[km/s]', 'vzCM[km/s]'][(np.where((binary['
       M1[M*]']>ranges[j-1]) & (binary['M1[M*]']<ranges[j])))]
8     m3=binary['dist[pc]', 'M2[M*]', 'L2[L*]', 'xCM[pc]', 'yCM[pc]', 'zCM
       [pc]', 'vxCM[km/s]', 'vyCM[km/s]', 'vzCM[km/s]'][(np.where((binary['
       M2[M*]']>ranges[j-1]) & (binary['M2[M*]']<ranges[j])))]
9     massdist.append(func.coupling(m1, m2, m3))
10
11 m1=single['dist[pc]', 'M[M*]', 'L[L*]', 'x[pc]', 'y[pc]', 'z[pc]', 'vx[
       km/s]', 'vy[km/s]', 'vz[km/s]'][(np.where((single['M[M*]']>ranges
       [-1])))]
12 m2=binary['dist[pc]', 'M1[M*]', 'L1[L*]', 'xCM[pc]', 'yCM[pc]', 'zCM[pc]
       ', 'vxCM[km/s]', 'vyCM[km/s]', 'vzCM[km/s]'][(np.where((binary['M1[M
       *]']>ranges[-1])))]
13 m3=binary['dist[pc]', 'M2[M*]', 'L2[L*]', 'xCM[pc]', 'yCM[pc]', 'zCM[pc]
       ', 'vxCM[km/s]', 'vyCM[km/s]', 'vzCM[km/s]'][(np.where((binary['M2[M
       *]']>ranges[-1])))]
14 massdist.append(func.coupling(m1, m2, m3))

```

Listing 14: Python script computing the list of objects in a binary systems or single in the right mass range.

We then obtain a list of list, *massdist*, with the information needed. Now, if we want to compute the distribution for objects in the first mass range, we write exactly the same thing as in Listing 13, as if there were only a single file, replacing the *lsingle* list by *massdist*[0].

And like in the first computation of distributions, we write the values in a file, one line per snapshot. This means that we have $(m + 4)$ times more files of datas. Among those files, we have the files with the values for half the objects of each parameter. In this one, one line is for one snapshot, and in a single line you have each value of different parameters.

We also want to plot other things, such as the histograms of the spatial distribution of CO. We write it as Listing 15. To know if there are binary or ternary systems, it is easier to write zeros when there are none. This way, we know the corresponding snapshot, because each one has 6 lines, in order.

```

1 if BINARY:
2     wd=[]
3     wweight=[]

```

```

4   Bwd=[]
5   Bweight=[]
6   for k in WD:
7       l1, w1, l2, w2 = func.distri_position_binary(k, single, N_single
, binary, N_binary)
8       wd=wd+l1
9       wweight=wweight+w1
10      Bwd=Bwd+l2
11      Bweight=Bweight+w2
12
13  NS, nweight, BNS, bnweight = func.distri_position_binary(13, single,
N_single, binary, N_binary)
14  BH, bweight, BBH, bbweight = func.distri_position_binary(14, single,
N_single, binary, N_binary)
15
16  hist=[Bwd, Bweight, BNS, bnweight, BBH, bbweight]
17  with open("../output2/lvl"+level+"/Hist_binary.dat", "a") as f:
18      for j in range(6):
19          if len(hist[j])>0:
20              for i in range(len(hist[j])):
21                  f.write(f"{hist[j][i]} ")
22                  f.write("\n")
23          else:
24              f.write(f"{0} \n")
25  else:
26      with open("../output2/lvl"+level+"/Hist_binary.dat", "a") as f:
27          for j in range(6):
28              f.write(f"{0} \n")

```

Listing 15: Python script computing the spatial distribution of each CO in a binary system

To write the histogram of masses, differentiated by type of objects, we just need to write the list of the masses of each type in a file. This is easy because we already created a list containing this type of informations for the distributions per type of objects.

We also want to fit the dependence of the value of the velocity dispersion to the mass of the object. To do that we wrote the Listing 16, using the `scipy.optimize.curve_fit` module.

```

1 ranges=[0.0, 0.2, 0.3, 0.4, 0.6, 0.9]
2 m=len(ranges)
3
4 sigma50=[]
5 sigma50.append([sigmass50[i][0] for i in range(m)])
6 sigma50.append([sigmass50[i][1] for i in range(m)])
7 sigma50.append([sigmass50[i][2] for i in range(m)])
8 massrange=[(ranges[i+1]-ranges[i])/2 + ranges[i] for i in range(m-1)]
9 massrange.append(1.0)
10
11 def affine(x, a, b):
12     return a*x+b
13
14 paramr, cov = curve_fit(affine, massrange, sigma50[0])
15 fitr=[affine(x, paramr[0], paramr[1]) for x in massrange]
16 paramp, cov = curve_fit(affine, massrange, sigma50[1])
17 fitp=[affine(x, paramp[0], paramp[1]) for x in massrange]

```

```

18 paramt, cov = curve_fit(affine, massrange, sigma50[2])
19 fitt=[affine(x, paramt[0], paramt[1]) for x in massrange]

```

Listing 16: Python script computing the affine fit of the velocity dispersion of half each population as a function of their mass

Finally, we want to plot the 3-dimensional distribution of the BH population. We only need the Cartesian coordinates of all the BH, and we decide to keep the information of the type of system in which the BH is. Once again, we need to write zeros if there are no BHs in the category of system to keep up with the number of the snapshot.

```

1 k=14
2 with open("../output2/lvl"+level+"/BH3D_binary.dat", "a") as f:
3     if BINARY:
4         l1binary=binary['xCM[pc]', 'yCM[pc]', 'zCM[pc]'][np.where((
5             binary['K*1']==k) & (binary['K*2']==k))]
6         l2binary=binary['xCM[pc]', 'yCM[pc]', 'zCM[pc]'][np.where((
7             binary['K*1']==k) & (binary['K*2']!=k))]
8         for j in range(3):
9             for i in range(len(l1binary)):
10                f.write(f"{l1binary[i][j]} ")
11            f.write("\n")
12        for j in range(3):
13            for i in range(len(l2binary)):
14                f.write(f"{l2binary[i][j]} ")
15            f.write("\n")
16    else:
17        for j in range(6):
18            f.write(f"{0} \n")

```

Listing 17: Python script computing the coordinates of BHs in a binary system

Plots

Once we have the data files, we can plot the distributions. We needed to have multiple scripts because the module can't open more than 20 plots at the same time. Therefore we separated the main plots, the plots per mass range, the plots per type of object, the histograms and the plot in 3-dimension, mainly because the scripts to draw them are similar. But they all have the same frame: first we verify which line corresponds to the snapshot we want to plot, then we read the files and put them in the lists that are adapted, and finally, we plot the different distributions we want.

For all this to work well (and to be able to go over all the snapshots we have in a script which is the goal), we must have, for each simulation, a text file with the number of each snapshot we have. Then, we can find the index corresponding to the snapshot we want to plot in the data files, in each plotting script, with the Listing 18.

```

1 with open("../Level_"+level+"/level"+level+"_snapshots.txt", 'r') as f:
2     lines=f.readlines()
3 test=False
4 aa=0
5 while test==False and aa<len(lines):
6     test=(lines[aa].split()[0]==time)
7     aa+=1

```

```
8 aa=aa-1
```

Listing 18: Python script finding the corresponding index of the snapshot number "time"

Then for each parameter we create a list containing the values from the data file we wrote in the script from annex 6.

```
1 with open("../output2/lvl"+level+"/Radii.dat", "r") as f:
2     liste=f.readlines()
3     line=liste[aa].split()
4     radii=[float(line[j]) for j in range(len(line))]
5 with open("../output2/lvl"+level+"/Half_values.dat", "r") as f:
6     halves=f.readlines()
7     half=halves[aa].split()
8     half_value=[float(half[j]) for j in range(len(half))]
```

Listing 19: Python script creating the list of radii and values for half the objects

With those lists, we can finally draw the plots and save them in a folder whose name is given as an input by the user. We made the choice to save the figures as *.jpg* files to save space. The plot drawn by the Listing 20 is the Figure 3 for the snapshot 46560 of the simulation of level A.

```
1 fig=plt.figure(clear=True)
2 plt.title("Mass density distribution of stars and remnants (level "+
3     level+")")
4 plt.plot(radii, mdensity)
5 plt.axhline(y = half_value[4])
6 plt.axvline(x = half_value[2])
7 plt.axvline(x = half_value[0], color='k')
8 plt.xscale("log")
9 plt.yscale("log")
10 plt.xlabel("distance from cluster's center (pc)")
11 plt.ylabel("mass density ($M_{\odot}/pc^{-3})")
12 plt.annotate(f'bins={half_value[18]}', xy=(0.8, 0.05), xycoords='axes
13     fraction')
14 fig.canvas.draw()
15 image_rgb = Image.fromarray(np.array(fig.canvas.renderer.buffer_rgba()))
16     .convert("RGB")
17 image_rgb.save(figfile+'densitymass.jpg', 'JPEG')
```

Listing 20: Python script plotting the distribution of mass density of stars and remnants, with a line for the half-mass radius, the Jacobi radius and the mass density of half the objects and showing the bins

There are some differences if we draw the plots separating the population in categories. There are, of course, loops over the mass ranges or type, to put the different curves on the same plot; like the list *massdist* in the script of annex 6, we have lists of lists for each parameter. To compare each subcategory, like in figure 5, we also loop inside a plot like in Listing 21. For each plot there is a half-mass radius, a value for half the objects of the sub-population and the binning of the curve shown in the same color.

```
1 color=['blue', 'purple', 'orange', 'green', 'red', 'pink']
2 x_fraction, y_fraction = 0.95, 0.05
3
4 fig=plt.figure(clear=True)
5 plt.title("mass density distribution of stars and remnants per mass
6     range (solar mass) (level "+level+")")
```

```

6 for j in range(m-1):
7     plt.plot(Radii[j], mdensity[j], label=f"{ranges[j]} $< M <$ {ranges[
8         j+1]}", color=color[j])
9     plt.axvline(x = half_value[j][1], color = color[j])
10    plt.axhline(y = half_value[j][2], color=color[j])
11    plt.annotate(f"bins={half_value[j][16]}", xy=(x_fraction, y_fraction
12        +j*0.04),
13        xycoords='axes fraction', ha='right', va='bottom',
14        color=color[j])
15    plt.plot(Radii[-1], mdensity[-1], label=f"{ranges[-1]} $< M$ ", color=
16        color[-1])
17    plt.axvline(x = half_value[m-1][1], color = color[-1])
18    plt.axhline(y = half_value[m-1][2], color=color[-1])
19    plt.annotate(f"bins={half_value[m-1][16]}", xy=(x_fraction, y_fraction+(
20        m-1)*0.04),
21        xycoords='axes fraction', ha='right', va='bottom',
22        color=color[-1])
23    plt.xscale("log")
24    plt.yscale("log")
25    plt.legend(bbox_to_anchor=(1.1, 1.0), loc='upper right')
26    plt.xlabel("distance from cluster's center (pc)")
27    plt.ylabel("mass density ($M_{\odot}/pc^{-3})")
28    fig.canvas.draw()
29    image_rgb = Image.fromarray(np.array(fig.canvas.renderer.buffer_rgba()))
30    .convert("RGB")
31    image_rgb.save(figfile+'densitymass_permass.jpg','JPEG')

```

Listing 21: Python script plotting the distributions of mass density of objects per mass range, with a line for the half-mass radius and the mass density of half the objects and showing the bins

When plotting the histograms of spatial dispersion of number and mass of each CO this is different. Here is an example with BHs in Listing 22.

```

1 with open("../output2/lvl"+level+"/Hist_single.dat", "r") as f:
2     lines=f.readlines()
3
4 bh=lines[6*aa+4].split()
5 BH=[float(bh[i]) for i in range(len(bh))]
6 Bweight=lines[6*aa+5].split()
7 bweight=[float(Bweight[i]) for i in range(len(Bweight))]
8
9 BINARY, MERGER = False, False
10
11 fig=plt.figure(clear=True)
12 plt.title("histogram of the distribution of BH (level "+level+")")
13 if len(BH)>0:
14     Bin=np.logspace(start=np.log10(min(BH)), stop=np.log10(max(BH)), num
15         =50)
16 else:
17     Bin=50
18 plt.hist(BH, bins=Bin, weights=bweight, label="distribution of black
19     holes")
20 with open("../output2/lvl"+level+"/Hist_binary.dat", "r") as f:
21     lines=f.readlines()
22 if float(lines[6*aa+4].split()[0])!=0:
23     BINARY=True

```

```

22     bbh=lines[6*aa+4].split()
23     BBH=[float(bbh[i]) for i in range(len(bbh))]
24     BBweight=lines[6*aa+5].split()
25     bbweight=[float(BBweight[i]) for i in range(len(BBweight))]
26     plt.hist(BBH, bins=Bin, weights=bbweight, label="distribution of
        black holes binaries")
27 with open("../output2/lvl"+level+"/Hist_merger.dat", "r") as f:
28     lines=f.readlines()
29 if float(lines[6*aa+4].split()[0])!=0:
30     MERGER=True
31     tbh=lines[6*aa+4].split()
32     TBH=[float(tbh[i]) for i in range(len(tbh))]
33     TBweight=lines[6*aa+5].split()
34     Tbweight=[float(TBweight[i]) for i in range(len(TBweight))]
35     plt.hist(TBH, bins=Bin, weights=Tbweight, label="distribution of
        black holes ternaries")
36 plt.xlabel("distance from cluster's center (pc)")
37 plt.ylabel("mass of BH ( $M_{\odot}$ )")
38 plt.legend()
39 fig.canvas.draw()
40 image_rgb = Image.fromarray(np.array(fig.canvas.renderer.buffer_rgba()))
    .convert("RGB")
41 image_rgb.save(figfile+'histmass_BH.jpg','JPEG')

```

Listing 22: Python script plotting the histogram of mass spatial dispersion of BH, separating the different systems

Comparison of snapshots

We can also draw those distributions with the same distribution from another snapshot. The idea of comparing 2 snapshots from the different simulations or every snapshot of one simulation is the same; we only change the files to read and the number of plots on the same figure. Another difference is that the comparison per time loop is done by itself over the number of snapshots in the file mentioned earlier, while when comparing 2 snapshots, the number of the snapshots is an input from the user.

But then it looks like the beginning of annex 6, and we verify the index of the snapshot, create the lists and plot. The exception is that the first two steps are done multiple times, or once for level A and once for level C, either once for each snapshot (done in a loop). The plots will look like the ones we got when comparing separate populations in annex 6, see listings 23 and 24.

```

1 fig=plt.figure(clear=True)
2 plt.title("mass distribution of stars and remnants")
3 plt.plot(Aradii, Amdensity, label="level A")
4 plt.plot(Cradii, Cmdensity, label="level C")
5 plt.xscale("log")
6 plt.yscale("log")
7 plt.legend()
8 plt.xlabel("distance from cluster's center (pc)")
9 plt.ylabel("mass density ( $M_{\odot}/pc^{-3}$ )")
10 fig.canvas.draw()
11 image_rgb = Image.fromarray(np.array(fig.canvas.renderer.buffer_rgba()))
    .convert("RGB")

```

```
12 image_rgb.save(figfile+'densitymass.jpg','JPEG')
```

Listing 23: Python script to plot the comparison of mass density distribution profile between level A and level C

```
1 x_fraction, y_fraction = 0.95, 0.05
2
3 fig=plt.figure(clear=True)
4 plt.title("Number density of stars and remnants per mass range (solar
    mass) (level "+level+"")")
5 for j in range(loop):
6     plt.plot(Radii[j], density[j], label= f"time= {Times[j]} (Mys)",
        color=color[j])
7     plt.axvline(x = half_value[j][1], color = color[j])
8     plt.axhline(y = half_value[j][3], color=color[j])
9     plt.annotate(f"bins={half_value[j][19]}", xy=(x_fraction, y_fraction
        +j*0.04),
10                xycoords='axes fraction', ha='right', va='bottom',
        color=color[j])
11 plt.xscale("log")
12 plt.yscale("log")
13 plt.legend(loc='best')
14 plt.xlabel("distance from cluster's center over half-mass radius")
15 plt.ylabel("density ($pc^{-3}$)")
16 fig.canvas.draw()
17 image_rgb = Image.fromarray(np.array(fig.canvas.renderer.buffer_rgba()))
    .convert("RGB")
18 image_rgb.save(figfile+'density.jpg','JPEG')
```

Listing 24: Python script to plot the comparison of mass density distribution profile between snapshots of level A

Time evolution

In the same idea as previously, we can plot the time evolution of some parameters, as listed in section 4. Those parameters are written in a file when running the script described in the annex 6, we just need to put them in a list and convert the number of the snapshot to a time in Myrs, thanks to parameters given in the global file of each level.

After getting the level the user wants to compare the snapshots from as an input, we create a list with the number of snapshots, and a list with the conversion in Myrs. Then we separately open three files to put the values in different lists; we have a file with the numbers and masses of the different CO systems, a file with the main informations on the cluster and the file with the values of parameters for half the population.

And we plot like done before the different figures shown section 4.

Looping on the snapshots

Finally we can run all those scripts, looping on a list of chosen snapshots with the following script. Because we add lines each time, we need to delete and recreate the folders containing the output files at the beginning of the code. Then we loop on the levels and read the file listing the number of the snapshots. We loop on this list of numbers, verifying we have a folder for the figures, and using the *subprocess* module to run the different

scripts described above. We added, at the beginning and the end, a print of the time to have an idea of the running time because it is a long process and it is interesting to know how much time it takes. For 14 snapshots in total, we have around 50 min of running time.

During this internship we used different versions of this script. For example, when running on HPC, we don't plot the figures for the ~ 4000 snapshots, we only create the datas. And then we import those data files on our environment and use a version that only plots the figures for the snapshots we want.

Follow an object through the snapshots

For the next part of the internship, we were interested in characterising binaries, therefore we wanted to see the evolution of the parameters of the system such as eccentricity, period or semi-major axis. It is easier to search in each snapshot one object that we know will be part of a binary system than a whole system. To also have a better understanding of the general evolution of said object we decided to write a text file with every information on the object for each snapshot, with the informations of its companion(s) and the system they form if its in a multiple system. This file is note aimed to be read by a script, but only by a user, to visually see, for example, the change of system the object is in.

In this script we first need to get the level of the simulation and the NAME of the object we want to follow. Then we create a list containing the number of every snapshot of this level to loop over. We also create lists of the indexes of the informations we want to write in our files, because depending on the level and type of system the object is in, the same information won't be at the same place in the table.

Then we loop over the snapshots. We create a table for each file existing (single, binary or ternary) and then look where, in those tables, the object is. When we find it, we write in the text file its informations. Then, depending if its in a multiple system, we write the same informations about its companion(s) and the informations on the general system. When its a ternary system, we write the informations of either the closest binary or the binary we were already following in the last snapshot. That is for the text file. For the data file, we simply use a list, at this step of the script, for each parameter we want to retrieve. Because we need the lists to have the same length, it has been arbitrary chosen to put a zero in the parameters of the binary when there are no binaries.

Some objects escape the cluster before the end of the simulation. When this happens, the code writes "escaped" in the text file and end the loop. We can then write the data file with each parameter in a column.